JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Enum Class ElementType

java.lang.Object
  java.lang.Enum<ElementType>
    java.lang.annotation.ElementType

All Implemented Interfaces:

Serializable, Comparable<ElementType>, Constable

```
public enum ElementType
extends Enum<ElementType>
```

The constants of this enumerated class provide a simple classification of the syntactic locations where annotations may appear in a Java program. These constants are used in `Target` meta-annotations to specify where it is legal to write annotations of a given type.

The syntactic locations where annotations may appear are split into *declaration contexts*, where annotations apply to declarations, and *type contexts*, where annotations apply to types used in declarations and expressions.

The constants `ANNOTATION_TYPE`, `CONSTRUCTOR`, `FIELD`, `LOCAL_VARIABLE`, `METHOD`, `PACKAGE`, `MODULE`, `PARAMETER`, `TYPE`, and `TYPE_PARAMETER` correspond to the declaration contexts in JLS §9.6.4.1.

For example, an annotation whose interface is meta-annotated with `@Target(ElementType.FIELD)` may only be written as a modifier for a field declaration.

The constant `TYPE_USE` corresponds to the type contexts in JLS §4.11, as well as to two declaration contexts: class and interface declarations (including annotation declarations) and type parameter declarations.

For example, an annotation whose interface is meta-annotated with `@Target(ElementType.TYPE_USE)` may be written on the class or interface of a field (or within the class or interface of the field, if it is a nested or parameterized class or interface, or array class), and may also appear as a modifier for, say, a class declaration.

The `TYPE_USE` constant includes class and interface declarations and type parameter declarations as a convenience for designers of type checkers which give semantics to annotation interfaces. For example, if the annotation interface `NonNull` is meta-annotated with `@Target(ElementType.TYPE_USE)`, then `@NonNull class C {...}` could be treated by a type checker as indicating that all variables of class `C` are non-null, while still allowing variables of other classes to be non-null or not non-null based on whether `@NonNull` appears at the variable's declaration.

See *Java Language Specification*:

9.6.4.1 @Target

4.1 The Kinds of Types and Values

Since:

1.5

Nested Class Summary

Nested classes/interfaces declared in class Enum
`Enum.EnumDesc<E>`

Enum Constant Summary

Enum Constants

Enum Constant	Description
`ANNOTATION_TYPE`	Annotation interface declaration (Formerly known as an annotation type.)
`CONSTRUCTOR`	Constructor declaration
`FIELD`	Field declaration (includes enum constants)
`LOCAL_VARIABLE`	Local variable declaration
`METHOD`	Method declaration
`MODULE`	Module declaration.
`PACKAGE`	Package declaration
`PARAMETER`	Formal parameter declaration
`RECORD_COMPONENT`	Record component
`TYPE`	Class, interface (including annotation interface), enum, or record declaration
`TYPE_PARAMETER`	Type parameter declaration
`TYPE_USE`	Use of a type

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
`static ElementType`	**`valueOf`**`(String name)`	Returns the enum constant of this class with the specified name.
`static ElementType[]`	**`values`**`()`	Returns an array containing the constants of this enum class, in the order they are declared.

Methods declared in class Enum

`clone`, `compareTo`, `describeConstable`, `equals`, `finalize`, `getDeclaringClass`, `hashCode`, `name`, `ordinal`, `toString`, `valueOf`

Methods declared in class Object

`getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Enum Constant Details

TYPE

```
public static final ElementType TYPE
```

Class, interface (including annotation interface), enum, or record declaration

FIELD

```
public static final ElementType FIELD
```

Field declaration (includes enum constants)

METHOD

```
public static final ElementType METHOD
```

Method declaration

PARAMETER

```
public static final ElementType PARAMETER
```

Formal parameter declaration

CONSTRUCTOR

`public static final ElementType CONSTRUCTOR`

Constructor declaration

LOCAL_VARIABLE

`public static final ElementType LOCAL_VARIABLE`

Local variable declaration

ANNOTATION_TYPE

`public static final ElementType ANNOTATION_TYPE`

Annotation interface declaration (Formerly known as an annotation type.)

PACKAGE

`public static final ElementType PACKAGE`

Package declaration

TYPE_PARAMETER

`public static final ElementType TYPE_PARAMETER`

Type parameter declaration

Since:

1.8

TYPE_USE

`public static final ElementType TYPE_USE`

Use of a type

Since:

1.8

MODULE

`public static final ElementType MODULE`

Module declaration.

Since:

9

RECORD_COMPONENT

```
public static final ElementType RECORD_COMPONENT
```

Record component

See *Java Language Specification*:

8.10.3 Record Members
9.7.4 Where Annotations May Appear

Since:

16

Method Details

values

```
public static ElementType[] values()
```

Returns an array containing the constants of this enum class, in the order they are declared.

Returns:

an array containing the constants of this enum class, in the order they are declared

valueOf

```
public static ElementType valueOf(String name)
```

Returns the enum constant of this class with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this class. (Extraneous whitespace characters are not permitted.)

Parameters:

`name` - the name of the enum constant to be returned.

Returns:

the enum constant with the specified name

Throws:

`IllegalArgumentException` - if this enum class has no constant with the specified name

`NullPointerException` - if the argument is null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.